

# Micro-Frontend Canvas

A practical tool for implementation decisions

# Before you start: are micro-frontends right for your context?

*Micro-frontends aren't for every project.*

*Before investing time in the canvas, ask yourself honestly: do we actually need micro-frontends?*

## Three reasons to consider micro-frontends

1. **Scaling teams as your team grows**, a monolithic frontend becomes a bottleneck. Merge conflicts pile up, coordination overhead multiplies, and release cycles slow down. Micro-frontends let teams own independent slices of the application, enabling faster, safer deployments without constant coordination.
2. **Aligning architecture with business**, micro-frontends are organized around business domains not around code reusability. This alignment lets teams build, deploy, and evolve features independently while staying connected to business goals.
3. **Reducing risk and improving resilience**, a single bug in a monolith can bring down the entire app. Micro-frontends isolate failures, so problems don't ripple across the system. Your frontend becomes more resilient, maintainable, and easier to troubleshoot.

## When micro-frontends may NOT be the right answer

Before committing, assess honestly:

### ✗ Fewer than 2-3 teams need to release independently?

If you have one or two people, you probably don't need MFEs.

### ✗ No dedicated platform/infrastructure team available?

MFEs require governance, shared shell management, and observability infrastructure. Without platform investment, you'll create chaos instead of autonomy.

### ✗ Immature automation pipelines (flaky tests, unreliable deployments)?

Micro-frontends amplify your deployment problems. Fix these first.

# What micro-frontends actually are

*Micro-frontends are the technical representation of a business subdomain. They allow independent implementations with the same or different technology, while minimizing shared code and owned by a single team.*

Key characteristics:

- **Context-aware:** Knows its domain, how it behaves, where it runs
- **Independent:** Built, tested, deployed without coordination
- **API-driven:** Clear input/output contracts or event definitions
- **Coarse-grained:** Covers a discrete business slices (checkout, catalog, reviews), not components (buttons, inputs)

**Critical distinction:** Micro-frontends optimize for independence, not reusability. Confusing them with components leads to coupling, brittle global states, and bottlenecks.

## Organizational Readiness

Micro-frontends aren't just technical, they're organizational.

Ask yourself:

- ***Do we have clear business domains that could map to teams?***
- ***Can we afford to invest in platform/infrastructure?***
- ***Are we ready to embrace distributed system thinking (contracts, resilience, failure isolation)?***
- ***Can we commit to governance guardrails to accelerate the development cycle (bundle limits, performance budgets, testing)?***

If the answer to any of these is "no" you might not ready yet.

That's okay.

Come back when you are.

# Micro-Frontend Canvas

A practical tool for implementation decisions

**The Micro-Frontend Canvas is a structured tool designed to capture the key implementation decisions for a single micro-frontend.**

It serves as a reference artifact that helps teams systematically address technical, organisational, and architectural concerns that will impact the micro-frontend's success.

More than a planning document, the canvas works as both a *planning tool* and an *assessment tool*:

**Planning Mode:** Before you write a single line of code, the canvas forces your team to ask critical questions about boundaries, constraints, dependencies, and quality standards. This prevents costly rework later and ensures alignment between engineering, product, and leadership before implementation begins.

**Assessment Mode:** If you already have micro-frontends (or suspect your architecture is struggling), the canvas becomes a diagnostic tool. Use it to audit your current system, identify where boundaries are leaking, surface hidden dependencies, pinpoint where responsibilities are unclear, and determine what needs to be restructured.

Either way, the canvas provides a shared language for discussing architecture complexity—something most organizations desperately need.

## How to use this Canvas

Before using this canvas, teams should conduct a workshop with cross-functional stakeholders, including engineers, engineering managers, architects, and product representatives (15-20 people maximum).

# Micro-Frontend Canvas

A practical tool for implementation decisions

During this workshop, identify the boundaries of the frontend application using techniques such as event storming or domain storytelling. Once boundaries are established, the canvas becomes the tool for documenting the technical details necessary for implementation.

The canvas also serves multiple purposes beyond initial planning:

- ***When friction emerges during development***, the canvas provides a framework for reassessing boundaries and validating whether your decisions still hold.
- ***When you need to formalise proposals***, the canvas ensures all key aspects of a micro-frontend are documented before presenting to stakeholders.
- ***When auditing existing architectures***, the canvas surfaces where decisions were made (or weren't) and where misalignment between teams has created problems.

## The fundamental principle

One micro-frontend, one canvas.

Each micro-frontend in your system should have its own dedicated canvas to document implementation decisions and validate boundaries. As your system evolves, create new iterations rather than overwriting previous versions. This maintains an audit trail of how your architecture evolved and why decisions changed.

## In modern development

In practices incorporating spec-driven development and context engineering, the canvas outputs serve as structured specifications for code assistants and AI-powered development tools.

A well-filled canvas provides the context that code generation needs to produce production-ready implementations.

# Micro-Frontend Canvas

A practical tool for implementation decisions

## When multiple Micro-Frontends share the same setup

If your micro-frontends share identical technical constraints, dependencies, or patterns, you don't need to repeat the same information across every canvas.

Create a shared baseline document (a wiki page, a Markdown file in your repository, or an internal standards page) that captures the common setup: technology stack, performance budgets, CI/CD patterns, design system version, and shared dependencies.

Individual canvases can then reference this baseline ("Same as organization standard") and focus on what's unique to each micro-frontend: its specific business capability, its boundaries, its risks, and its communication contracts.

This keeps each canvas focused on differentiation rather than repetition.

## What you'll find here

This guide walks you through each section of the canvas with:

- Guiding questions that help you think through critical decisions
- Concrete examples from an e-commerce domain
- A blank template ready for printing and workshop use

The canvas emerged from years of hands-on work with engineering teams across consulting engagements, real-world delivery, and discussions at international conferences. The same patterns, and the same mistakes, surfaced repeatedly, regardless of industry or scale.

This canvas distills those lessons into a practical tool that helps teams move from architectural intent to production-ready decisions with clarity and confidence.

# Micro-Frontend Canvas

A practical tool for implementation decisions

## How much detail to include

The A4 constraint is intentional: it forces conciseness. Each section should contain enough information to make the decision clear, but not so much that the canvas becomes a specification document.

A good rule: if you can't summarize a section in 3-5 bullet points or 2-3 sentences, your micro-frontend's scope may be too broad, or you're capturing implementation details that belong in your README instead.

The canvas should be scannable in under 5 minutes by someone unfamiliar with the project. If reading it takes longer, you've included too much.

Resist the temptation to shrink fonts or cram additional detail. If a section feels too small for what you need to capture, that's a signal worth paying attention to: either your boundary needs rethinking, or the detail belongs in a different artifact.

## Using the Canvas for the first time

If this is your first time using the canvas, it's normal to feel uncertain about what level of detail each section requires. Start with the sections you can answer confidently (business purpose, dependencies, technical constraints) and work toward the ones that require more discussion (boundaries validation, communication methodology, governance).

The canvas is designed to surface gaps in your thinking. If a section is difficult to fill, that's valuable information: it means you haven't made that decision yet, and you should.

Consider working through the canvas with a colleague or your tech lead the first time. After one or two uses, the format becomes intuitive.

# Micro-Frontend Canvas

A practical tool for implementation decisions

## License

This work is licensed under the [Creative Commons Attribution–NonCommercial 4.0 International License](https://creativecommons.org/licenses/by-nc/4.0/) (CC BY-NC 4.0).

***You are free to use, share, and adapt this material for non-commercial purposes, provided appropriate credit is given.***

## Attribution

© Luca Mezzalana.

*Originally published as part of the Micro-Frontend Canvas.*

Adaptations must clearly indicate that they are derived from the original Micro-Frontend Canvas and must not imply endorsement by the original author.



# Systems details, Team ownership & Workshop information

## Overview

This section establishes the foundational context for the micro-frontend: which system it belongs to, which team owns it, and the workshop details for tracking iterations over time.

## Guiding Questions

- Which software system does this micro-frontend belong to?
- Which team has end-to-end ownership (design, development, deployment, operations)?
- When was this canvas created or last updated?
- What iteration number is this (for tracking evolution)?

## Example

*Software System:* ShopNow E-commerce Platform

*Owned by Team:* Team Checkout

*Workshop Date:* January 15, 2026

*Iteration:* 2

The highlighted area shows where this section appears in the Micro-Frontend Canvas.

|                                                                                                                                                                                             |                                                                                                           |                                                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Micro-Frontend description</b> <i>Name and brief description: what does this MFE do? What user journey/business capability does it enable?</i>                                           |                                                                                                           | <b>Split Strategy</b> <i>Horizontal or Vertical split.</i>                                                                                                           |
| <b>Boundaries Validation</b><br><i>Verify the micro-frontend has minimal API surface, is context-aware and self-contained, coarse-grained in nature, and can be deployed independently.</i> | <b>Dependencies</b><br><i>External dependencies such as shared libraries, application shell, or APIs.</i> | <b>Communication Methodology</b><br><i>How data flows into and out of the micro-frontend (e.g., event emitter, web storage, query strings).</i>                      |
| <b>Organisational Constraints</b><br><i>Any organisational requirement that limits the freedom of decision.</i>                                                                             | <b>Technical Constraints</b><br><i>Any technical requirement that restricts the freedom of decision.</i>  | <b>Composition Context</b><br><i>How is this micro-frontend composed and integrated into the application (e.g., CSR or SSR, module federation, transclusion).</i>    |
| <b>Challenges &amp; Risks</b><br><i>Identify current challenges and risks.</i>                                                                                                              |                                                                                                           | <b>Quality Guardrails</b><br><i>Define the three most important guardrails to implement (e.g. bundle size limits, performance budgets, accessibility standards).</i> |

# Micro-Frontend details

## Overview

Defines what the micro-frontend does from a business and user perspective. This section articulates the purpose and scope, ensuring clarity on the value delivered.

## Guiding Questions

- What is the name of this micro-frontend?
- What business capability does it enable?
- What user journey does it support?
- What problem does it solve for users?

## Example

*Micro-Frontend Description:*

Checkout Flow - Enables users to review cart items, apply discount codes, select shipping options, enter payment details, and complete purchase.

The highlighted area shows where this section appears in the Micro-Frontend Canvas.

# Micro-Frontend Canvas

Software System

Owned by Team

Workshop Date

Iteration no.

## Micro-Frontend details

Micro-Frontend description: name and brief description: what does this MFE do? What user journey/business capability does it enable?

Split Strategy Horizontal or Vertical split.

### Boundaries Validation

Verify the micro-frontend has minimal API surface, is context-aware and self-contained, coarse-grained in nature, and can be deployed independently.

### Dependencies

External dependencies such as shared libraries, application shell, or APIs.

### Communication Methodology

How data flows into and out of the micro-frontend (e.g., event emitter, web storage, query strings).

### Organisational Constraints

Any organisational requirement that limits the freedom of decision.

### Technical Constraints

Any technical requirement that restricts the freedom of decision.

### Composition Context

How is this micro-frontend composed and integrated into the application (e.g., CSR or SSR, module federation, transclusion).

### Challenges & Risks

Identify current challenges and risks.

### Quality Guardrails

Define the three most important guardrails to implement (e.g. bundle size limits, performance budgets, accessibility standards).

# Architectural & Organisational decisions

## Overview

This section captures high-level architectural decisions about how the micro-frontend is structured and integrated, along with organisational factors that constrain implementation choices.

The split strategy determines whether the micro-frontend employs a horizontal split (multiple micro-frontends in the same view) or vertical split (one micro-frontend per view or group of views).

This decision affects team coordination, integration complexity, and deployment independence.

Boundaries validation verifies that the micro-frontend is correctly decomposed using four critical criteria: minimal API surface, context awareness, appropriate granularity, and deployment independence.

Organisational constraints identify requirements that limit implementation freedom, such as budget, timeline, team structure, compliance requirements, or organisational standards that must be followed.

## Guiding Questions

### *Split Strategy:*

- Will this micro-frontend share the view with other micro-frontends (horizontal) or own entire routes (vertical)?
- What level of coordination with other teams is required?

### *Boundaries Validation:*

- Does the micro-frontend expose fewer than 5 props to its container?
- Can it function independently with minimal external context?
- Is it coarse-grained rather than component-sized?
- Can it be deployed without coordinating with other teams?

# Architectural & Organisational decisions

## Guiding Questions

### *Organisational Constraints:*

- Are there budget or timeline constraints?
- Are there specific compliance or regulatory requirements?
- Are there organisational standards or methodologies that must be followed?
- What resources are available to the team?

### *Challenges & Risks:*

- What technical uncertainties exist?
- What organizational risks could impact delivery?
- What could cause this micro-frontend to fail?
- What dependencies represent risk?
- How will you mitigate each identified risk?

## Example

*Split Strategy:* Vertical split - The checkout micro-frontend owns all routes under /checkout/\* exclusively.

*Boundaries Validation:* Exposes only `userId` and `cartId` props to container. Fetches all checkout data independently via GraphQL APIs. Represents complete checkout flow from cart review to order confirmation. Deployed independently through dedicated CI/CD pipeline without coordination.

*Organisational Constraints:* Must complete implementation within Q1 2026; PCI-DSS compliance required for payment handling; must use company-approved AWS services only; maximum team size of 5 engineers allocated.

*Challenges & Risks:* Team owns checkout but depends on cart team for pricing logic, creating a hidden coupling that slows both teams

The highlighted area shows where this section appears in the Micro-Frontend Canvas.

# Micro-Frontend Canvas

Software System

Owned by Team

Workshop Date

Iteration no.

|                                                                                                                                                                                                         |                                                                                                                       |                                                                                                                                                                                  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Micro-Frontend description <i>Name and brief description: what does this MFE do? What user journey/business capability does it enable?</i>                                                              |                                                                                                                       | Split Strategy <i>Horizontal or Vertical split.</i>                                                                                                                              |
| <div>Boundaries Validation</div> <div><i>Verify the micro-frontend has minimal API surface, is context-aware and self-contained, coarse-grained in nature, and can be deployed independently.</i></div> | <div>Dependencies</div> <div><i>External dependencies such as shared libraries, application shell, or APIs.</i></div> | <div>Communication Methodology</div> <div><i>How data flows into and out of the micro-frontend (e.g., event emitter, web storage, query strings).</i></div>                      |
| <div>Organisational Constraints</div> <div><i>Any organisational requirement that limits the freedom of decision.</i></div> <div>Architectural &amp; Organizational Decisions</div>                     | <div>Technical Constraints</div> <div><i>Any technical requirement that restricts the freedom of decision.</i></div>  | <div>Composition Context</div> <div><i>How is this micro-frontend composed and integrated into the application (e.g., CSR or SSR, module federation, transclusion).</i></div>    |
| <div>Challenges &amp; Risks</div> <div><i>Identify current challenges and risks.</i></div>                                                                                                              |                                                                                                                       | <div>Quality Guardrails</div> <div><i>Define the three most important guardrails to implement (e.g. bundle size limits, performance budgets, accessibility standards).</i></div> |

# Technical decisions

## Overview

This section documents the technical implementation choices, constraints, and validation criteria that define how the micro-frontend will be built and integrated into the broader system.

The composition context specifies whether the micro-frontend uses client-side, or server-side, or edge-side composition, and what integration mechanism enables loading. Communication methodology defines how data flows in and out, including input mechanisms and event patterns.

Dependencies capture external requirements including shared libraries, APIs, and application shell capabilities.

Technical constraints identify technology stack requirements and architectural patterns that must be followed.

Quality guardrails establish measurable standards for performance, bundle size, accessibility, and architecture compliance. Challenges and risks surface known technical uncertainties and potential failure points requiring mitigation strategies.

## Guiding Questions

### *Composition Context:*

- Is composition client-side, or server-side?
- What technology is used for integration (Module Federation, transclusion, web components)?

### *Communication Methodology:*

- What data does the micro-frontend receive as input?
- How does it communicate with other micro-frontends?
- What are the explicit inputs and outputs to this MFE?
- What's the versioning strategy if contracts change?

### *Dependencies:*

- What shared libraries or packages does this micro-frontend use?
- How are shared dependencies versioned and upgraded?
- What APIs does it consume?
- What does it require from the application shell?



# Technical decisions

## Technical Constraints:

- *What technology stack must be used?*
- *Are there platform or infrastructure limitations?*
- *What architectural patterns must be followed?*
- *How do you monitor this MFE's performance in production? (RUM, synthetic)*
- *What metrics matter most? (response time, error rate, user impact)*
- *What's your alerting strategy?*

## Quality Guardrails:

- *What is the maximum acceptable bundle size?*
- *What performance metrics must be met?*
- *What accessibility standard must be achieved?*
- *How will architectural boundaries be enforced?*

## Example

*Composition Context:* CSR rendering using Module Federation.

*Communication Methodology:* Reads auth token from cookie; emits checkout:completed event with orderId. Uses event emitter pattern.

*Dependencies:* Application shell provides React 18.2, React Router 6.x

*Technical Constraints:* React 18+ and TypeScript 5+; AWS CloudFront/S3 deployment; max bundle 300KB.

*Quality Guardrails:* 1) Bundle max 300KB gzipped, code-split by route; 2) Lighthouse >90, LCP <2.5s; 3) ArchUnit tests for boundary enforcement.

The highlighted area shows where this section appears in the Micro-Frontend Canvas.

# Micro-Frontend Canvas

Software System

Owned by Team

Workshop Date

Iteration no.

|                                                                                                                                                                                             |                                                                                                                                                                      |                                                                                                                                                                   |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Micro-Frontend description</b> <i>Name and brief description: what does this MFE do? What user journey/business capability does it enable?</i>                                           |                                                                                                                                                                      | <b>Split Strategy</b> <i>Horizontal or Vertical split.</i>                                                                                                        |
| <b>Boundaries Validation</b><br><i>Verify the micro-frontend has minimal API surface, is context-aware and self-contained, coarse-grained in nature, and can be deployed independently.</i> | <b>Dependencies</b><br><i>External dependencies such as shared libraries, application shell, or APIs.</i>                                                            | <b>Communication Methodology</b><br><i>How data flows into and out of the micro-frontend (e.g., event emitter, web storage, query strings).</i>                   |
| <b>Organisational Constraints</b><br><i>Any organisational requirement that limits the freedom of decision.</i>                                                                             | <b>Technical Constraints</b><br><i>Any technical requirement that restricts the freedom of decision.</i>                                                             | <b>Composition Context</b><br><i>How is this micro-frontend composed and integrated into the application (e.g., CSR or SSR, module federation, transclusion).</i> |
| <b>Challenges &amp; Risks</b><br><i>Identify current challenges and risks.</i>                                                                                                              | <b>Quality Guardrails</b><br><i>Define the three most important guardrails to implement (e.g. bundle size limits, performance budgets, accessibility standards).</i> |                                                                                                                                                                   |

## Technical Decisions

# Real-world example: the ShopNow story

## The Challenge

ShopNow is a mid-sized e-commerce platform serving customers across Europe and North America. After five years of growth, their monolithic frontend had become a bottleneck. A single codebase served by one large team meant every feature release required careful coordination, and deployment anxiety was at an all-time high.

The business demanded faster delivery. Marketing wanted rapid A/B testing. Product needed independent feature releases. Engineering was drowning in merge conflicts and regression bugs.

## The Workshop

In January 2026, ShopNow convened a three-day event storming workshop with 18 participants: engineers from the existing frontend team, backend engineers, product managers, UX designers, and the engineering leadership.

Over those three days, they mapped the entire customer journey and identified 5 distinct bounded contexts:

- Homepage
- Catalog
- Cart
- Checkout
- Account

The company decided to split the monolith into 5 autonomous teams aligned with these domains.

## The Canvas Session

Two weeks after event storming, Team Checkout gathered for a half-day workshop to complete their Micro-Frontend Canvas. They had clarity on what their domain was, but needed to nail down how it would work technically. The architectural and organizational decisions needed to be documented before writing the first line of code.

*What follows is their completed canvas, the artifact that guided their implementation and became their source of truth when questions arose during development.*

|                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                         |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>Micro-Frontend description</div> <div>CHECKOUT FLOW - HANDLES COMPLETE ORDER SUBMISSION FROM CART REVIEW TO CONFIRMATION. INCLUDES ORDER SUMMARY, SHIPPING SELECTION, PAYMENT PROCESSING, AND CONFIRMATION SCREENS.</div>                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                   | <div>Split Strategy</div> <div>VERTICAL SPLIT - OWNS ALL /CHECKOUT/* ROUTES EXCLUSIVELY.</div>                                                                                                                                                                                                                                                                                          |
| <div>Boundaries Validation</div> <div>RECEIVES ONLY USERID AND CARTID PROPS.</div> <div>FETCHES CHECKOUT DATA VIA GRAPHQL APIS. REPRESENTS COMPLETE CHECKOUT FLOW.</div> <div>DEPLOYED INDEPENDENTLY VIA DEDICATED CI/CD PIPELINE.</div>                                                                                                                                                            | <div>Dependencies</div> <div>SHELL PROVIDES REACT 18.2, REACT ROUTER 6.X, AUTH CONTEXT, EVENT EMITTER.</div> <div>APIS:<ul style="list-style-type: none"><li>PAYMENT GATEWAY V3,</li><li>SHIPPING SERVICE V2,</li><li>DISCOUNT SERVICE V1.</li></ul></div> <div>DESIGN SYSTEM V4.1</div> <div>STRIPE SDK V3</div> | <div>Communication Methodology</div> <div>INPUT:<ul style="list-style-type: none"><li>CARTID FROM ROUTE PARAMS,</li><li>AUTH TOKEN FROM COOKIE</li></ul></div> <div>OUTPUT:<ul style="list-style-type: none"><li>EMITS CHECKOUT:STARTED,</li><li>CHECKOUT:COMPLETED (WITH ORDERID, ORDERTOTAL),</li><li>CHECKOUT:ABANDONED</li></ul></div> <div>EVENTS VIA SHELL'S EVENT EMITTER.</div> |
| <div>Organisational Constraints</div> <div>Q1 2026 DELIVERY DEADLINE FOR PEAK SEASON.</div> <div>PCI-DSS LEVEL 1 COMPLIANCE REQUIRED.</div> <div>AWS-ONLY INFRASTRUCTURE.</div> <div>5-ENGINEER TEAM LIMIT.</div> <div>MUST USE EXISTING PAYMENT GATEWAY.</div> <div>EU LEGAL REVIEW REQUIRED FOR TERMS DISPLAY.</div>                                                                              | <div>Technical Constraints</div> <div>REACT 18+ AND TYPESCRIPT 5+.</div> <div>AWS CLOUDFRONT/S3 DEPLOYMENT.</div> <div>300KB BUNDLE LIMIT.</div> <div>OAUTH 2.0 AUTH VIA SHELL.</div> <div>BFF PATTERN FOR APIS.</div> <div>2-SECOND RESPONSE SLA.</div> <div>CSP HEADERS MANDATORY.</div>                        | <div>Composition Context</div> <div>CLIENT-SIDE RENDERING WITH MODULE FEDERATION V2.</div> <div>LOADED BY APPLICATION SHELL ON /CHECKOUT NAVIGATION.</div> <div>RUNS AS STANDALONE REACT APP WITHIN SHELL CONTAINER.</div>                                                                                                                                                              |
| <div>Challenges &amp; Risks</div> <div>STRIPE SDK 85KB REQUIRES LAZY LOADING. PAYMENT GATEWAY 95% SLA NEEDS RETRY LOGIC AND FALLBACK.</div> <div>SHELL DEPLOYMENT COORDINATION REQUIRES SEMANTIC VERSIONING.</div> <div>Q2 PCI AUDIT MAY REQUIRE ARCHITECTURE CHANGES.</div> <div>THIRD-PARTY ADDRESS VALIDATION NEEDS CIRCUIT BREAKER.</div> <div>10X HOLIDAY TRAFFIC REQUIRES LOAD TESTING.</div> |                                                                                                                                                                                                                                                                                                                   | <div>Quality Guardrails</div> <ul style="list-style-type: none"><li>BUNDLE MAX 300KB WITH CODE-SPLITTING BY ROUTE. LAZY LOAD STRIPE SDK ON PAYMENT STEP.</li><li>LIGHTHOUSE &gt;90, LCP &lt;2.5S, FID &lt;100MS, CLS &lt;0.1. RUM MONITORING IN PRODUCTION.</li><li>ARCHUNIT TESTS ENFORCE NO CIRCULAR DEPENDENCIES, VALIDATE BFF PATTERN, VERIFY MODULE BOUNDARIES.</li></ul>          |

# Blank Micro-Frontend Canvas template

## Overview

This page contains a blank Micro-Frontend Canvas ready for printing and use in workshops.

**Remember the fundamental rule: one micro-frontend, one canvas.**

Each micro-frontend in your system should have its own dedicated canvas to document implementation decisions and validate boundaries.

Print multiple copies for collaborative workshop sessions where teams can work together to define and refine their micro-frontend architecture.

**For best results, print this page in A4 landscape format.**

Use the canvas during workshops with engineers, architects, and stakeholders to collaboratively document decisions before implementation begins.

## Feedback and Contributions

Your feedback helps improve this tool for the entire community.

If you have suggestions, encounter issues, or want to propose changes to the canvas:

- Email: [mezzalab@gmail.com](mailto:mezzalab@gmail.com)
- GitHub: [\[github.com/your-repo/micro-frontend-canvas\]](https://github.com/your-repo/micro-frontend-canvas)

**The canvas is freely available under a Creative Commons license on GitHub.**

Feel free to open issues, or share how you've adapted it for your organisation's needs.

This will help the Micro-Frontend Canvas to evolve and be useful to many other organisations around the world!

# Micro-Frontend Canvas

Software System

Owned by Team

Workshop Date

Iteration no.

|                            |                       |                           |
|----------------------------|-----------------------|---------------------------|
| Micro-Frontend description |                       | Split Strategy            |
| Boundaries Validation      | Dependencies          | Communication Methodology |
| Organisational Constraints | Technical Constraints | Composition Context       |
| Challenges & Risks         |                       | Quality Guardrails        |

# Next steps & resources

## After completing your canvas

With your Micro-Frontend Canvas complete, you have documented the critical implementation decisions for your micro-frontend.

The next steps include:

1. **Validate with stakeholders:** Review the canvas with engineering leadership, architects, and product teams to ensure alignment
2. **Create Architecture Decision Records (ADRs):**  
Transform key canvas decisions into formal ADRs for your documentation
3. **Prepare governance plan:** Establish processes for maintaining quality guardrails, dependency management, and architectural compliance
4. **Coordinate with impacted teams:** Schedule meetings with teams whose micro-frontends or systems will interact with yours to align on contracts and communication patterns
5. **Begin implementation:** Use the canvas as a specification for development, including context for AI-assisted coding tools

## Continuous evolution

The canvas is a living document.

Return to it when:

- Business requirements shift significantly
- New features challenge existing boundaries
- Technical constraints change
- Team friction emerges during implementation
- Retrospectives reveal architectural issues

Create new iterations rather than overwriting previous versions. This maintains an audit trail of architectural evolution.

# Next steps & resources

## Learn More

For deeper exploration of micro-frontend architecture and implementation patterns:

- **Building Micro-Frontends (2nd Edition)** - Comprehensive guide to micro-frontend architecture, patterns, and practices
- **buildingmicrofrontends.com** - Articles, case studies, video podcast, and community resources

## Community & Support

Join the conversation and share your experiences:

- Email: [mezzalab@gmail.com](mailto:mezzalab@gmail.com)
- GitHub: [github.com/your-repo/micro-frontend-canvas] - Report issues, propose improvements, or contribute examples

## About the author

**Luca Mezzalira** is a solutions architect specialising in frontend architectures, distributed systems, and cloud infrastructure.

Over the years, he has helped 100s of teams worldwide successfully adopt micro-frontend architecture, from Fortune 500 companies to fast-growing startups.

As the author of Building Micro-Frontends (O'Reilly), Luca has synthesised years of practical experience into frameworks and patterns that help organisations scale their frontend development.

LinkedIn/X: Follow for updates on micro-frontend practices and patterns



# Thank you